

SUBMITTED BY:

NAME: MARIHA MALIK
ROLLNO: 23-BCS-060
SECTION: A
SESSION: 2023-2027

SUBMITTED TO:

INSTRUCTOR: RESPECTED Mr. WAQAS SALEEM
COURSE TITLE: DEVOPS

SUBMISSION DATE:

OCTOBER 24, 2026

Table of Contents

Lab Tasks & Sections	Page
Lab Overview and Objectives	**3**
Task 1: Fork and Clone the Databases Repository	**3**
Task 2: Install Docker, Docker Compose, and Python on Ubuntu	**4**
Task 3: Start the Database Services	**5**
Task 4: MySQL with Values in the Code	**6**
Task 5: MongoDB with Values in the Code	**6**
Task 6: MongoDB with os.getenv()	**7**
Task 7: MongoDB with a Credentials File	**9**
DataBases Link:	**11**
Summary & Conclusion	**11**

Task 1: Fork and Clone the Databases Repository

Objective: Open Databases repository, select Fork, keep the name Databases, confirm GitHub repository structure, and clone your own fork into ~/Databases.

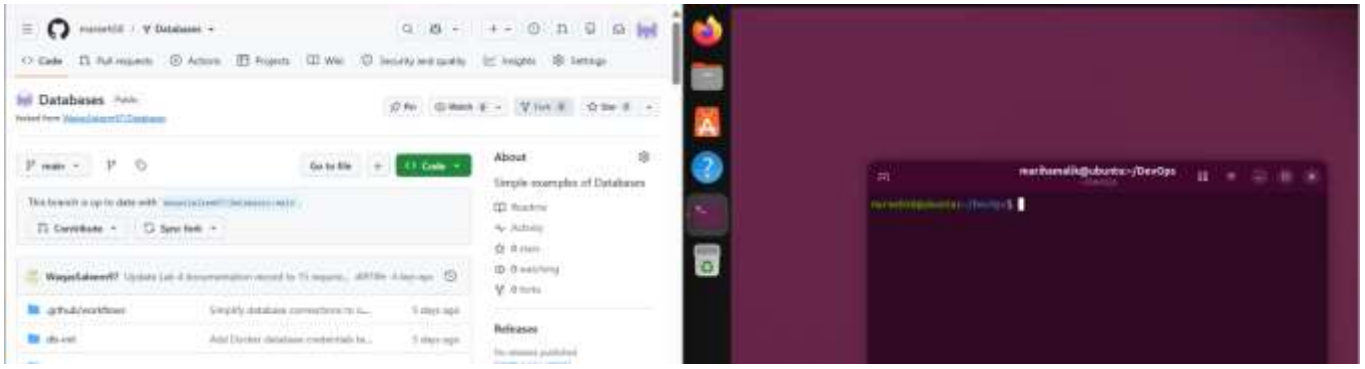
Commands Executed:

```
cd ~

git clone git@github.com:mareeh58/Databases.git

cd Databases ls -la
```

Verification Evidence:



databases_fork.png



databases_clone.png

Task 2: Install Docker, Docker Compose, and Python on Ubuntu
Objective: Update Ubuntu packages, install Docker Engine, docker-compose plugin, configure user group membership, install Python venv/pip, and verify installations.

Commands Executed:

```
sudo apt update -y
sudo apt install -y docker.io curl
sudo systemctl start docker
 sudo usermod -aG docker $USER
 git --version
docker –version
 docker compose version
 sudo apt install -y python3 python3-venv python3-pip
cd ~/Databases
python3 -m venv .venv
source .venv/bin/activate
pip install pymysql pymongo
```

Verification Evidence:



docker_installation

Task 3: Start the Database Services

Objective: Launch all four containerized databases via Docker Compose and inspect host ports mapped for MySQL and MongoDB instances.

Commands Executed:

```
docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
docker compose ps
docker compose port mysql 3306
docker compose port mongo-dev 27017
docker compose port mongo-test 27018
docker compose port mongo-prod 27019
```

Verification Evidence:

```
mareeh58@ubuntu:~/Databases$ sudo docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
[+] up 4/4
 ✔ Container databases-mongo-dev-1  Healthy                                    1.2s
 ✔ Container databases-mongo-test-1 Healthy                                    1.2s
 ✔ Container databases-mongo-prod-1 Healthy                                    1.2s
 ✔ Container databases-mysql-1      Healthy                                    6.1s
mareeh58@ubuntu:~/Databases$ sudo docker compose ps
NAME                   IMAGE       COMMAND                  SERVICE      CREATED          STATUS
    PORTS
databases-mongo-dev-1    mongo:7.0   "docker-entrypoint.s…"   mongo-dev    3 minutes ago    Up 3 minutes (healthy)
    127.0.0.1:27017->27017/tcp
databases-mongo-prod-1   mongo:7.0   "docker-entrypoint.s…"   mongo-prod   3 minutes ago    Up 3 minutes (healthy)
    127.0.0.1:27019->27017/tcp
databases-mongo-test-1   mongo:7.0   "docker-entrypoint.s…"   mongo-test   3 minutes ago    Up 3 minutes (healthy)
    127.0.0.1:27018->27017/tcp
databases-mysql-1        mysql:8.4   "docker-entrypoint.s…"   mysql        30 minutes ago   Up 16 seconds (healthy
)   33060/tcp, 127.0.0.1:3307->3306/tcp
mareeh58@ubuntu:~/Databases$
```

database_services

```
mareeh58@ubuntu:~/Databases$ sudo docker compose port mysql 3306
127.0.0.1:3307
mareeh58@ubuntu:~/Databases$ sudo docker compose port mongo-dev 27017
127.0.0.1:27017
mareeh58@ubuntu:~/Databases$ sudo docker compose port mongo-test 27017
127.0.0.1:27018
mareeh58@ubuntu:~/Databases$ sudo docker compose port mongo-prod 27017
127.0.0.1:27019
mareeh58@ubuntu:~/Databases$
```

database_ports.png

Task 4: MySQL with Values in the Code

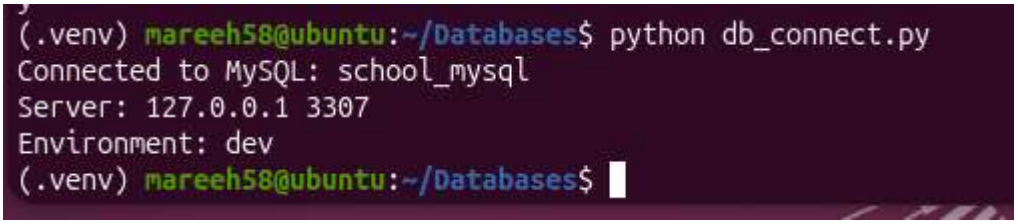
Objective: Create db_connect.py with hardcoded MySQL connection parameters, execute query against environment_info table, and verify connection output.

Commands Executed:

```
nano db_connect.py

python db_connect.py
```

Verification Evidence:



mysql_hardcoded_connection

Task 5: MongoDB with Values in the Code

Objective: Replace db_connect.py contents with MongoClient hardcoded connection string targeting school_dev database on port 27017.

Commands Executed:

```
nano db_connect.py

python db_connect.py
```

Verification Evidence:



mongo_hardcoded_connection

Task 6: MongoDB with os.getenv()

Objective: Update db_connect.py to load credentials dynamically via os.getenv() and test across dev, test, and prod environment variables.

Commands Executed:

```
export MONGO_USERNAME=dev_user
export MONGO_PASSWORD='DevDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27017
export MONGO_DATABASE=school_dev
clear
python db_connect.py
export MONGO_USERNAME=test_user
export MONGO_PASSWORD='TestDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27018
export MONGO_DATABASE=school_test
clear
python db_connect.py
export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='ProdDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
clear
python db_connect.py
```

Verification Evidence:



mongo_env_dev

```
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_USERNAME=dev_user
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_PASSWORD='DevDemoOnly123!'
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_ENDPOINT=127.0.0.1:27017
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_DATABASE=school_dev
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_USERNAME=test_user
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_PASSWORD='TestDemoOnly123!'
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_ENDPOINT=127.0.0.1:27018
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_DATABASE=school_test
(.venv) mareeh58@ubuntu:~/Databases$ python db_connect.py
connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) mareeh58@ubuntu:~/Databases$
```

mongo_env_test

```
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_USERNAME=prod_user
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_PASSWORD='ProdDemoOnly123!'
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_ENDPOINT=127.0.0.1:27019
(.venv) mareeh58@ubuntu:~/Databases$ export MONGO_DATABASE=school_prod
(.venv) mareeh58@ubuntu:~/Databases$ python db_connect.py
connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) mareeh58@ubuntu:~/Databases$
```

mongo_env_prod.png

Task 7: MongoDB with a Credentials File (Files & Permissions)
Objective: Create local credentials.ini configuration file containing dev, test, and prod sections, and restrict permissions using chmod 600.

Commands Executed:

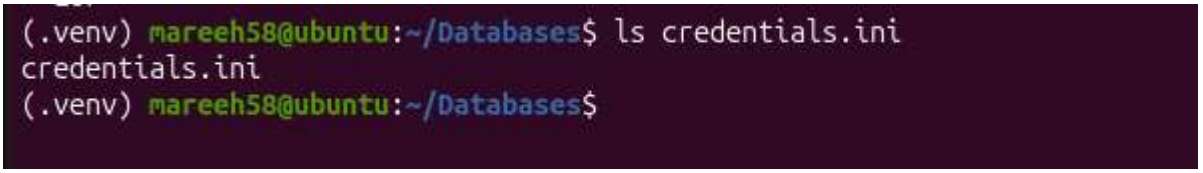
```
cp -n credentials.ini.example credentials.ini

ls credentials.ini

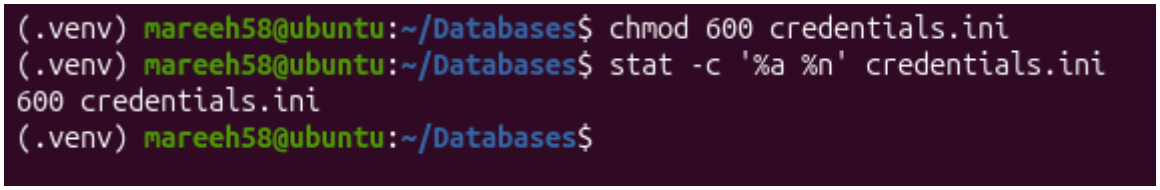
chmod 600 credentials.ini

stat -c '%a %n' credentials.ini
```

Verification Evidence:



credential_files_created



credential_file_permissions.png

Task 7: MongoDB with a Credentials File (Execution & Verification)
Objective: Update db_connect.py to parse credentials.ini using configparser and execute script across APP_ENV for dev, test, and prod environments.

Commands Executed:

```
export APP_ENV=dev

python db_connect.py

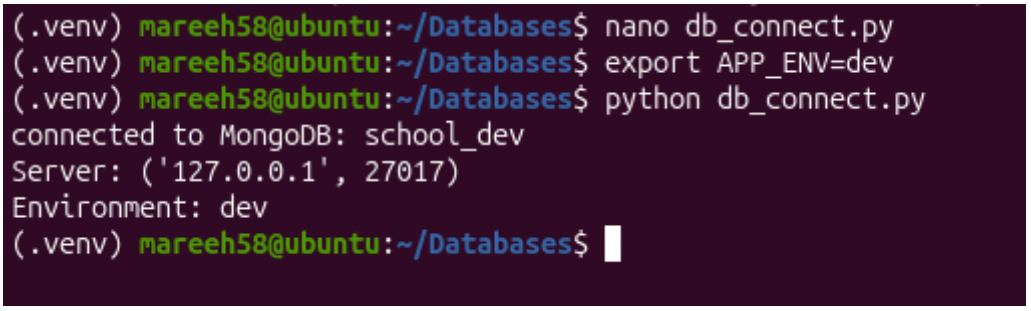
export APP_ENV=test

python db_connect.py

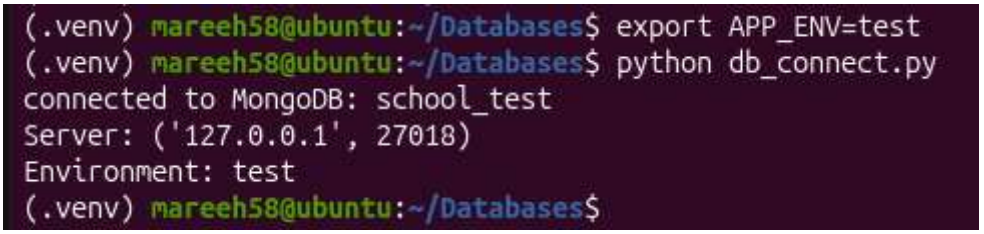
export APP_ENV=prod

python db_connect.py
```

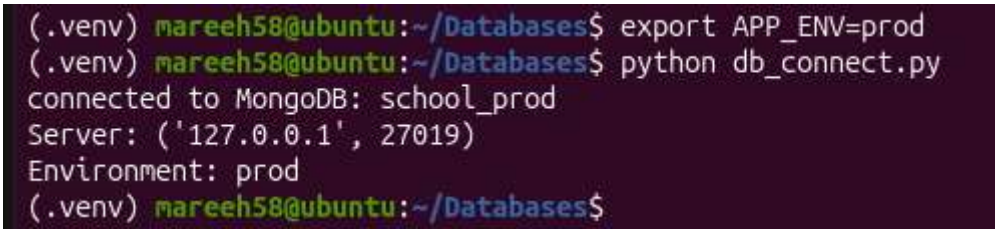
Verification Evidence:



mongo_file_dev



mongo_file_test.png



mongo_file_prod.png

DataBases Link:

Databases fork url

Summary & Conclusion

In Lab 4, you successfully implemented robust database connectivity scripts in Python for relational MySQL and document-oriented MongoDB instances running inside Docker containers. You progressed from hardcoded connection strings to environment variables, and ultimately utilized secure INI configuration files parsed via configparser.